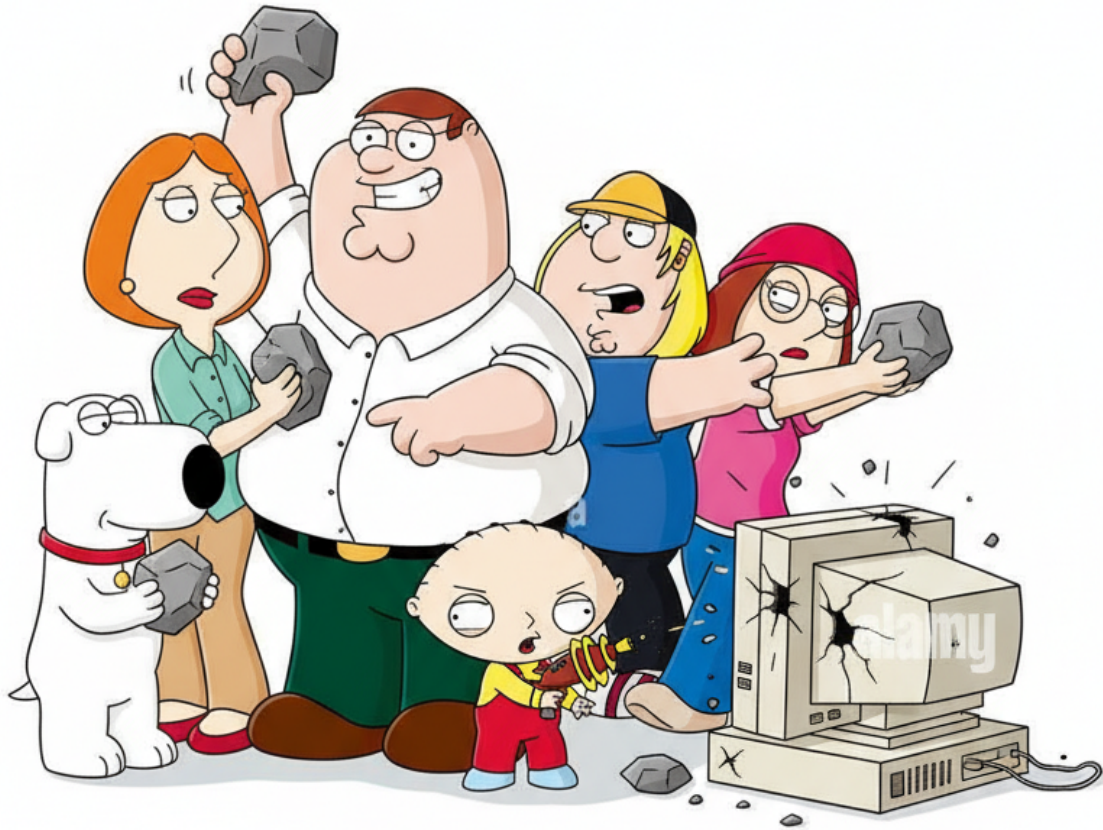




I LOVE CYBERCRIME



Università Politecnica delle Marche

Facoltà di Ingegneria

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e
dell'Automazione

Analisi e Predizione di Incidenti di Cybersecurity

Classificazione e Sviluppo di un Modello Predittivo

Docenti

Prof. Ursino Domenico
Dott. Buratti Christopher

Componenti del gruppo

Dott. Tempera Fabio
Dott. Marcianesi Luca
Dott. Vianello Gabriele

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	3
1.1	Obiettivi	3
2	Explorative Data Analysis	5
2.1	Panoramica del Dataset	5
2.2	Struttura e Gerarchia dei Dati	5
2.3	Analisi Descrittiva	8
2.3.1	Distribuzione della Variabile Target (IncidentGrade) .	8
2.3.2	Analisi Temporale degli Incidenti	9
2.3.3	Correlazione tra le Variabili Numeriche	10
2.3.4	Distribuzione delle Gravità degli Incidenti	11
2.3.5	Analisi Temporale degli Incidenti	12
2.3.6	Tecniche MITRE ATT&CK più Frequenti	12
2.4	Strumenti Utilizzati	13
3	Extract - Transform - Load	15
3.1	Preparazione del Dataset	15
3.2	Pulizia e Gestione dei Dati	16
3.2.1	Eliminazione delle Colonne	16
3.2.2	Rinominazione delle Colonne	16
3.2.3	Gestione dei Valori Nulli	16
3.3	Feature Engineering	17
3.3.1	Elaborazione della Colonna delle Tecniche MITRE .	17
3.3.2	Codifica delle Variabili Categoricali	17
3.3.3	Scaling delle Feature Numeriche	18
3.4	Il Problema dello Sbilanciamento	18
3.4.1	Bilanciamento Manuale	18
3.4.2	Sotto-campionamento e SMOTE	19
3.5	Risultati della Fase di Preprocessing	19
4	Classificazione	21
4.1	Metodologia di Valutazione	21

4.2	Modelli di Classificazione	22
4.2.1	Decision Tree	22
4.2.2	XGBoost	24
4.2.3	Random Forest	26
4.2.4	Multi-layer Perceptron	28
4.3	Valutazione Comparativa dei Modelli	32
4.4	Analisi Dettagliata delle Performance	33
4.5	Analisi di Interpretabilità: Feature Importance	34
4.6	Ensemble: analisi dell'Accordo tra Modelli	35
4.6.1	Modelli Ensemble: XGBoost & Random Forest e XG- Boost & MLP	35
4.7	Valutazione Finale del Modello	36
5	Conclusioni	38
5.1	Riepilogo e Interpretazione dei Risultati	38
5.1.1	La Sfida dei "Benign Positive"	38
5.1.2	Dinamicità del Dominio e Concept Drift	39
5.2	Contributi Principali del Progetto	39
5.3	Sviluppi Futuri e Prospettive di Ricerca	40

Elenco delle figure

2.1	Distribuzione delle tre classi di IncidentGrade.	8
2.2	Distribuzione di IncidentGrade in formato binario (TruePositive vs Non-TruePositive).	9
2.3	Distribuzione degli incidenti per giorno della settimana, suddivisi per IncidentGrade.	10
2.4	Matrice di correlazione tra le variabili numeriche del dataset.	11
2.5	Distribuzione della gravità degli incidenti.	12
2.6	Numero di incidenti generati nel corso del tempo.	12
2.7	Le 15 tecniche MITRE ATT&CK più frequenti nel dataset.	13
3.1	Matrice di correlazione (heatmap) tra le principali feature numeriche dopo il preprocessing. L'assenza di colori intensi (vicini a +1 o -1) indica una bassa correlazione, suggerendo che le feature portano informazioni relativamente indipendenti.	20
4.1	Metriche di classificazione dell'albero decisionale.	23
4.2	Matrice di confusione dell'albero decisionale.	23
4.3	Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).	24
4.4	Metriche di classificazione di XGBoost.	25
4.5	Matrice di confusione di XGBoost.	26
4.6	Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).	26
4.7	Metriche di classificazione della Random Forest.	27
4.8	Matrice di confusione della Random Forest.	28
4.9	Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).	28
4.10	Metriche di classificazione del Multi-layer Perceptron.	30
4.11	Matrice di confusione del Multi-layer Perceptron.	31
4.12	Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).	31

4.13	Confronto delle performance dei quattro modelli su cinque metriche chiave: Accuracy, Precision, Recall, F1-Score e ROC AUC. Si osserva come i modelli basati su alberi (Random Forest, XGBoost) tendano a sovraperformare il modello MLP di base.	32
4.14	Confronto dei punteggi ROC AUC per i modelli binari. XGBoost emerge come il modello con la più alta capacità discriminante (AUC = 0.9304), seguito da vicino da Random Forest e Decision Tree.	33
4.15	Matrici di confusione per i modelli Random Forest, Decision Tree e MLP. I valori indicano il numero di predizioni corrette e scorrette. La percentuale di recall per ogni classe ($TP / (TP + FN)$) è indicata in rosso. Si nota come tutti i modelli siano molto efficaci nell'identificare i casi Non-TP, ma il Random Forest mostra il miglior recall sulla classe TP (72.2%). . . .	33
4.16	Confronto delle 10 feature più importanti secondo i modelli XGBoost, Random Forest e Decision Tree. Sebbene ogni modello dia un peso leggermente diverso, emergono pattern comuni.	34
4.17	Analisi delle feature più comuni nella Top 10 di tutti e tre i modelli a albero.	34
4.18	Matrice di correlazione tra le predizioni dei modelli.	35
4.19	Confronto delle performance dei modelli in termini di F1-Score e ROC AUC.	35
4.20	Valori finali delle metriche ensemble	36

1 Introduzione

In questo lavoro sono state adottate diverse soluzioni tecniche, per esplorare, analizzare, modellare e classificare i dati contenuti in un dataset che contiene incidenti sulla sicurezza informatica.

1.1 Obiettivi

Gli obiettivi principali di questo progetto sono la progettazione, lo sviluppo e la valutazione di tecniche di machine learning per la classificazione e previsione di incidenti di cybersecurity. E' stata posta una maggiore attenzione sulla capacità dei modelli nel distinguere i falsi attacchi o eventi non pericolosi da rischi o attacchi reali. L'analisi si basa sul dataset GUIDED (Microsoft), la più grande raccolta pubblicamente disponibile di incidenti di cybersecurity del mondo reale, che offre un'opportunità unica per addestrare e testare modelli su scenari realistici.

Nello specifico, il progetto si articola nei seguenti obiettivi:

- **Esplorazione:** la fase di *Explorative Data Analysis* sarà fondamentale per avere piena comprensione dei dati e delle relazioni tra essi; in questo modo si selezioneranno le caratteristiche più importanti ai fini dell'analisi.
- **Classificazione:** creazione di modelli predittivi in grado di stimare con elevata accuratezza i gradi di valutazione degli incidenti — vero positivo (TP), benigno positivo (BP) e falso positivo (FP) — sulla base delle risposte storiche degli analisti e delle caratteristiche telemetriche degli avvisi. L'obiettivo è automatizzare il primo livello di analisi, consentendo agli analisti di concentrare le proprie risorse sulle minacce più critiche e riducendo il tempo medio di risposta.
- **Valutazione:** stabilire un criterio di valutazione standardizzato e riproducibile per i sistemi di risposta guidata utilizzando dati reali. Questo permetterà di valutare e confrontare oggettivamente le capacità dei modelli sviluppati.

- **Analisi interpretativa:** Esplorare, attraverso tecniche di interpretabilità dei modelli (es. feature importance), i fattori e le caratteristiche che contribuiscono maggiormente alla classificazione di un incidente. Questo obiettivo mira a fornire insight azionabili per i team di sicurezza, rispondendo a domande strategiche come: "Quali tipi di avvisi o tecniche MITRE ATT&CK sono più spesso associati a minacce reali?" o "Quali indicatori telemetrici sono più predittivi di un falso positivo?".

Attraverso l'impiego di queste tecniche, il progetto mira a fornire contributi significativi per una migliore comprensione delle minacce alla sicurezza e per la definizione di interventi di risposta personalizzati ed efficaci. L'obiettivo finale è ridurre il carico di lavoro manuale degli analisti SOC, minimizzare l'errore umano e, in ultima analisi, migliorare la postura di sicurezza complessiva delle organizzazioni.

2 Explorative Data Analysis

2.1 Panoramica del Dataset

Nel contesto della sicurezza informatica, la disponibilità di dati realistici e su larga scala è fondamentale per lo sviluppo e la validazione di nuove tecnologie difensive. A tal fine, Microsoft ha rilasciato **GUIDE (Guided Response Utterance and Incident Data at Enterprise-scale)**, la più grande raccolta pubblicamente disponibile di incidenti di cybersecurity del mondo reale.

GUIDE contiene oltre 13 milioni di prove per 33 tipi di entità, coprendo 1,6 milioni di avvisi e 1 milione di incidenti annotati con etichette di triage dai clienti in un periodo di due settimane. Di questi incidenti, 26.000 contengono etichette aggiuntive relative alle azioni di rimedio intraprese. La telemetria proviene da oltre 6.100 organizzazioni, con 9.100 `DetectorId` unici, personalizzati e integrati in numerosi prodotti di sicurezza, che coprono 441 tecniche uniche del framework MITRE ATT&CK.

L'elevata disponibilità dei dati presenti in GUIDE consente di addestrare modelli con tecniche del Machine e Deep Learning per implementare un efficiente sistema di triage degli avvisi.

2.2 Struttura e Gerarchia dei Dati

Il dataset è organizzato secondo una gerarchia a tre livelli che rispecchia il processo di aggregazione delle informazioni in un ambiente SOC: (1) prove (evidence), (2) avvisi (alerts) e (3) incidenti (incidents).

- **Prove:** Rappresentano il livello più granulare di informazione. Una prova è un singolo artefatto o evento atomico, come un indirizzo IP, un hash di file, un'e-mail o i dettagli di un processo. Ogni prova contiene metadati specifici che la contestualizzano.
- **Avvisi:** Al livello intermedio, gli avvisi consolidano una o più prove correlate per segnalare un potenziale evento di sicurezza. Ad esempio,

un avviso di "Esecuzione di malware sospetto" potrebbe aggregare prove relative a un hash di file, al processo che lo ha eseguito e alle connessioni di rete generate.

- **Incidenti:** Al livello più alto, gli incidenti comprendono uno o più avvisi, rappresentando una narrazione coesa di una potenziale violazione della sicurezza. È a questo livello che l'analista applica l'etichetta di triage finale — *Vero Positivo (TP)*, *Benigno Positivo (BP)* o *Falso Positivo (FP)* — che costituisce la variabile target del nostro studio.

La Tabella 2.1 riassume le colonne principali utilizzate in questo progetto, estratte e aggregate dai diversi livelli del dataset per creare un set di dati unificato a livello di incidente.

Colonna	Formato	Descrizione
Id	Stringa/Intero	Identificativo univoco del record o dell'evento.
OrgId	Stringa	Identificativo univoco dell'organizzazione.
IncidentId	Intero	Identificativo univoco dell'incidente.
AlertId	Stringa	Identificativo univoco dell'avviso.
Timestamp	Datetime	Data e ora dell'evento o della creazione dell'avviso.
DetectorId	Stringa	ID del rilevatore che ha generato l'avviso.
AlertTitle	Stringa	Titolo descrittivo dell'avviso.
Category	Categoriale	Categoria dell'avviso (es. Credential Access, Malware).
MitreTechniques	Lista di stringhe	Tecniche MITRE ATT&CK associate all'incidente.
IncidentGrade	Categoriale	Grado o classificazione dell'incidente.
ActionGrouped	Categoriale	Azione aggregata o raggruppata intrapresa (es. Blocked).
ActionGranular	Stringa	Azione specifica e dettagliata intrapresa.
EntityType	Categoriale	Tipo di entità coinvolta (es. File, Process, IpAddress).
EvidenceRole	Categoriale	Ruolo dell'entità come prova (es. Source, Destination).
DeviceId	Stringa	Identificativo univoco del dispositivo.
Sha256	Stringa (Hash)	Hash SHA256 di un file o di un'altra entità.
IpAddress	Stringa	Indirizzo IP coinvolto nell'evento.
Url	Stringa	URL coinvolto nell'evento.
AccountSid	Stringa	Security Identifier (SID) dell'account utente.

Colonna	Formato	Descrizione
AccountUpn	Stringa	User Principal Name (UPN) dell'account (es. utente@dominio.com).
AccountObjectId	Stringa	Object ID dell'account in Azure Active Directory.
AccountName	Stringa	Nome dell'account utente.
DeviceName	Stringa	Nome del dispositivo (hostname).
NetworkMessageId	Stringa	Identificativo univoco di un messaggio di rete (es. email).
EmailClusterId	Stringa/Intero	ID di un gruppo di email simili o correlate.
RegistryKey	Stringa	Percorso completo di una chiave di registro di Windows.
RegistryValueName	Stringa	Nome del valore all'interno di una chiave di registro.
RegistryValueData	Stringa	Dati contenuti in un valore di registro.
ApplicationId	Stringa	Identificativo univoco di un'applicazione.
ApplicationName	Stringa	Nome dell'applicazione.
OAuthApplicationId	Stringa	ID di un'applicazione OAuth coinvolta.
ThreatFamily	Stringa	Famiglia di minacce o malware associata (es. Zloader).
FileName	Stringa	Nome del file coinvolto.
FolderPath	Stringa	Percorso della cartella contenente il file.
ResourceIdName	Stringa	Nome di una risorsa (es. in un provider cloud).
ResourceType	Stringa	Tipo di risorsa.
Roles	Lista di stringhe	Ruoli associati a un'entità (es. utente, risorsa).
OSFamily	Categoriale	Famiglia del sistema operativo (es. Windows, Linux).
OSVersion	Stringa	Versione specifica del sistema operativo.
AntispamDirection	Categoriale	Direzione del flusso email (es. Inbound, Outbound).
SuspicionLevel	Categoriale	Livello di sospetto associato all'evento.
LastVerdict	Stringa	L'ultimo verdetto o classificazione dell'evento.
CountryCode	Stringa (ISO)	Codice ISO del paese associato (es. a un IP).
State	Stringa	Stato o regione associata.
City	Stringa	Città associata.

2.3 Analisi Descrittiva

L'analisi descrittiva è stata condotta per esplorare e sintetizzare le principali caratteristiche del dataset, fornendo una comprensione preliminare delle variabili, delle loro distribuzioni e delle relazioni reciproche. Questo passaggio è fondamentale per identificare pattern, anomalie e per guidare le successive fasi di pre-processing e modellazione.

2.3.1 Distribuzione della Variabile Target (IncidentGrade)

La variabile target, `IncidentGrade`, definisce la natura di un incidente e può assumere tre valori principali: `BenignPositive`, `TruePositive` e `FalsePositive`. La sua distribuzione è un aspetto cruciale, poiché uno sbilanciamento tra le classi può influenzare le performance e la generalizzazione dei modelli di classificazione. Come illustrato nella Figura 2.1, il dataset mostra un certo grado di sbilanciamento. La classe `BenignPositive` è la più rappresentata con il 43.4% del totale, seguita da `TruePositive` con il 35.1% e infine da `FalsePositive` con il 21.5%. Sebbene non estremo, questo squilibrio suggerisce che un modello potrebbe tendere a favorire la classe maggioritaria.

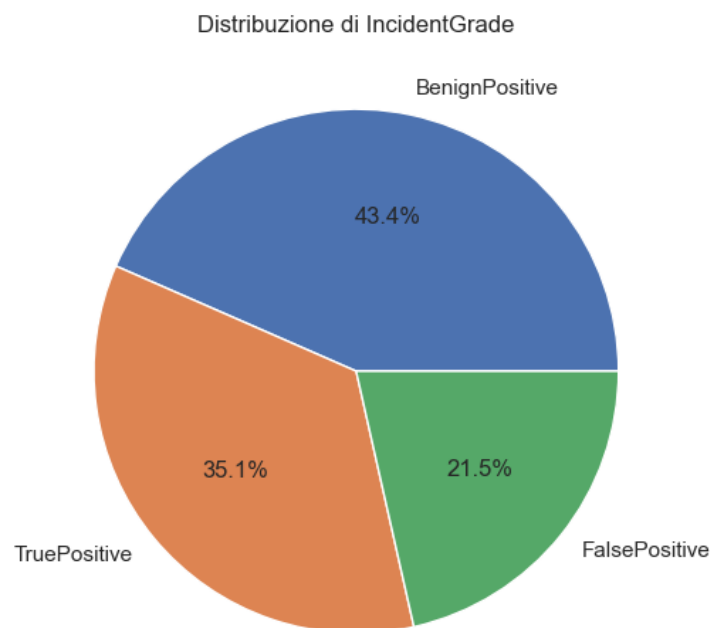


Figura 2.1: Distribuzione delle tre classi di IncidentGrade.

Per molti scenari applicativi, l'obiettivo primario è distinguere i veri positivi (`TruePositive`) da tutti gli altri eventi non rilevanti (`Non-TruePositive`,

che aggregano `BenignPositive` e `FalsePositive`). Riformulando il problema in un'ottica binaria, otteniamo la distribuzione mostrata in Figura 2.2. In questa configurazione, i `TruePositive` costituiscono il 40% del dataset, mentre i `Non-TruePositive` rappresentano il restante 60%. Questo sbilanciamento è meno pronunciato e offre una base più stabile per un compito di classificazione binaria.

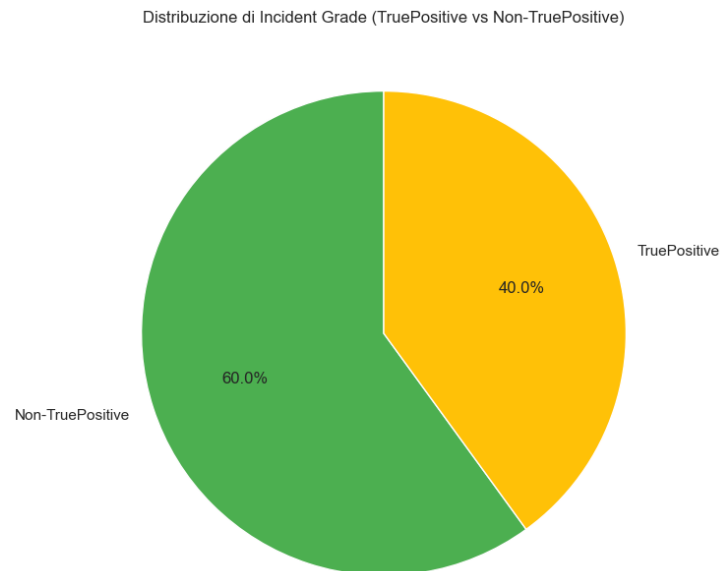


Figura 2.2: Distribuzione di IncidentGrade in formato binario (`TruePositive` vs `Non-TruePositive`).

2.3.2 Analisi Temporale degli Incidenti

L'analisi della distribuzione temporale degli incidenti può rivelare pattern legati alle attività operative. La Figura 2.3 mostra il numero totale di eventi registrati per ogni giorno della settimana, con una scomposizione basata sulla classe di `IncidentGrade`. Emerge chiaramente una concentrazione degli incidenti durante i giorni lavorativi, da lunedì a venerdì, con un picco massimo registrato il martedì. Al contrario, durante il fine settimana (sabato e domenica) si osserva una drastica riduzione del volume di eventi. Questa tendenza è prevedibile e riflette i cicli di attività aziendale. È interessante notare come la proporzione tra le diverse classi di incidenti rimanga relativamente costante durante la settimana lavorativa.

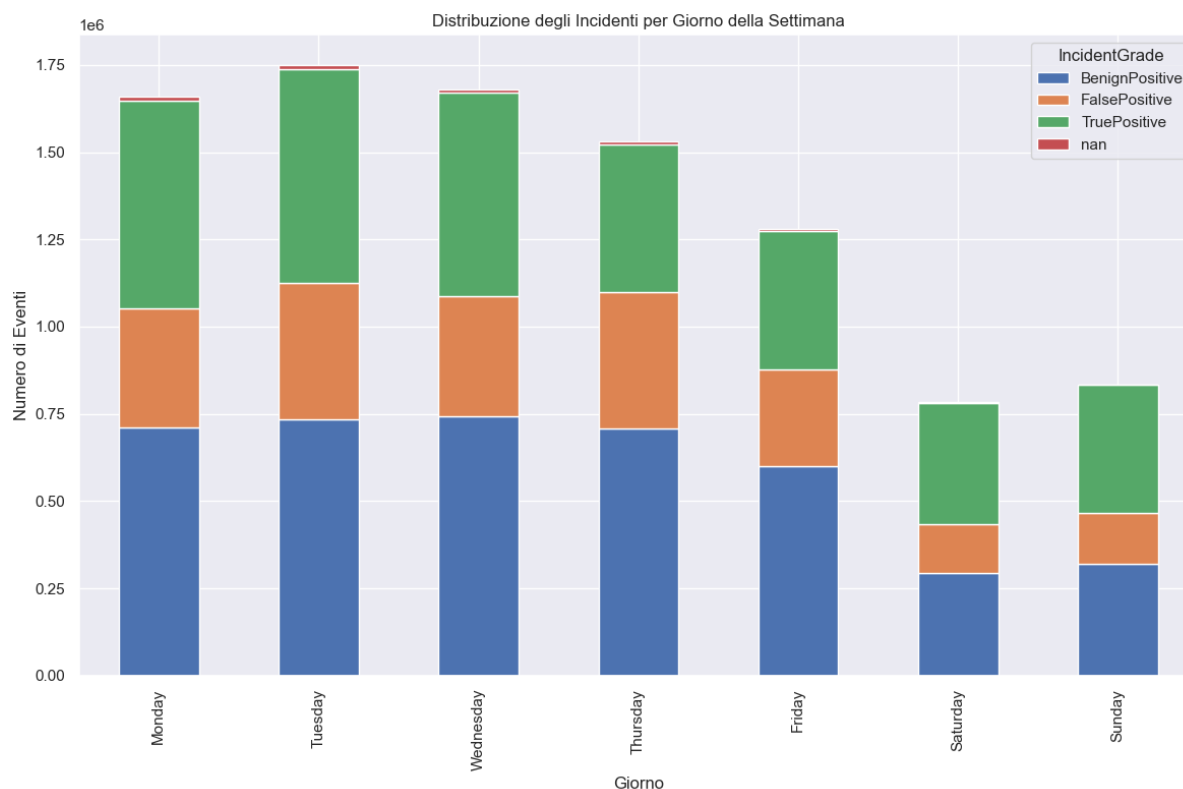


Figura 2.3: Distribuzione degli incidenti per giorno della settimana, suddivisi per IncidentGrade.

2.3.3 Correlazione tra le Variabili Numeriche

Per investigare le relazioni lineari tra le variabili numeriche del dataset, è stata calcolata e visualizzata una matrice di correlazione, mostrata in Figura 2.4. La matrice utilizza una scala di colori dove il rosso indica una forte correlazione positiva (valori vicini a +1), il blu una forte correlazione negativa (valori vicini a -1) e il bianco un'assenza di correlazione lineare (valori vicini a 0). Dall'analisi emergono alcuni cluster di variabili fortemente correlate. Ad esempio, si nota un'alta correlazione positiva tra AccountSid, AccountUpn e AccountObjectId, il che è atteso poiché sono tutti identificatori univoci dello stesso account utente. Allo stesso modo, variabili come FileName e FolderPath mostrano una correlazione significativa. L'identificazione di queste ridondanze è utile in fase di feature selection per evitare problemi di multicollinearità e per ridurre la dimensionalità del modello.

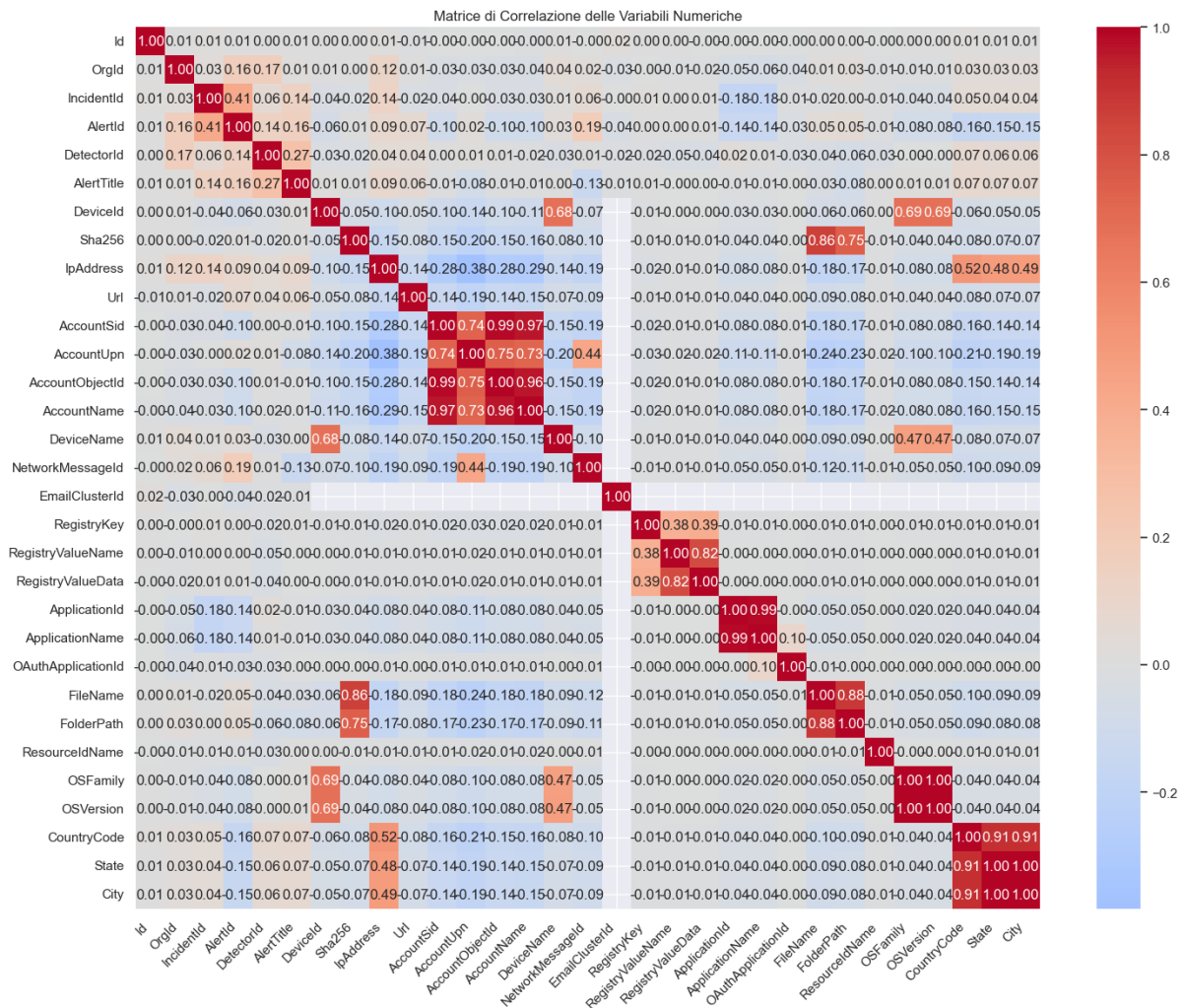


Figura 2.4: Matrice di correlazione tra le variabili numeriche del dataset.

2.3.4 Distribuzione delle Gravità degli Incidenti

L'analisi della gravità (Severity) degli incidenti, visualizzata in Figura 2.5, mostra la ripartizione tra le categorie Bassa, Media, Alta e Informativa. Questa distribuzione aiuta a comprendere il profilo di rischio generale degli incidenti analizzati e può essere correlata con la classificazione di triage per identificare pattern utili.

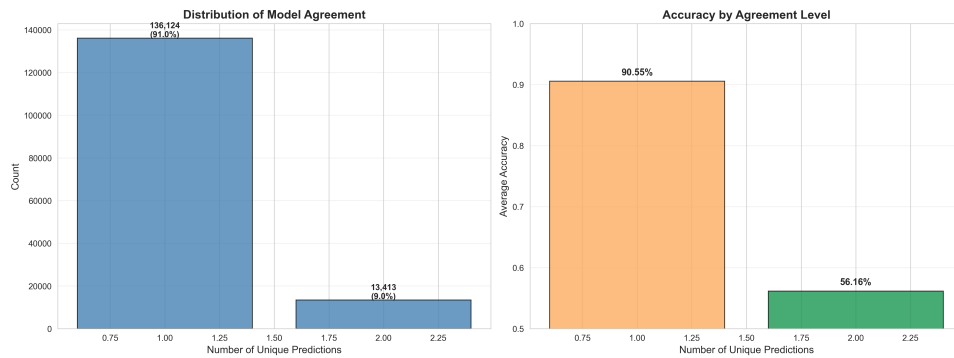


Figura 2.5: Distribuzione della gravità degli incidenti.

2.3.5 Analisi Temporale degli Incidenti

La visualizzazione del numero di incidenti nel tempo (Figura 2.6) permette di identificare eventuali trend o pattern ciclici, come picchi di attività durante i giorni lavorativi o in orari specifici. Questa analisi è fondamentale per comprendere le dinamiche temporali delle minacce e può fornire feature predittive importanti.

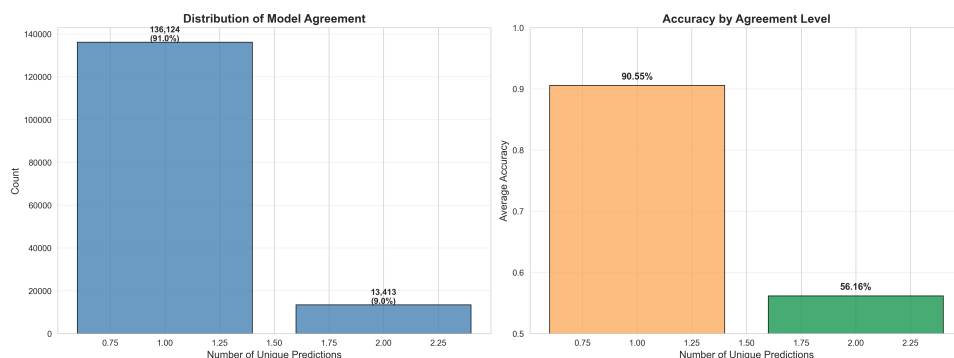


Figura 2.6: Numero di incidenti generati nel corso del tempo.

2.3.6 Tecniche MITRE ATT&CK più Frequenti

L'analisi delle tecniche MITRE ATT&CK (Figura 2.7) evidenzia le tattiche e le procedure più comunemente osservate negli incidenti. Identificare le tecniche prevalenti è cruciale per prioritizzare gli sforzi di difesa e comprendere il modus operandi degli avversari.

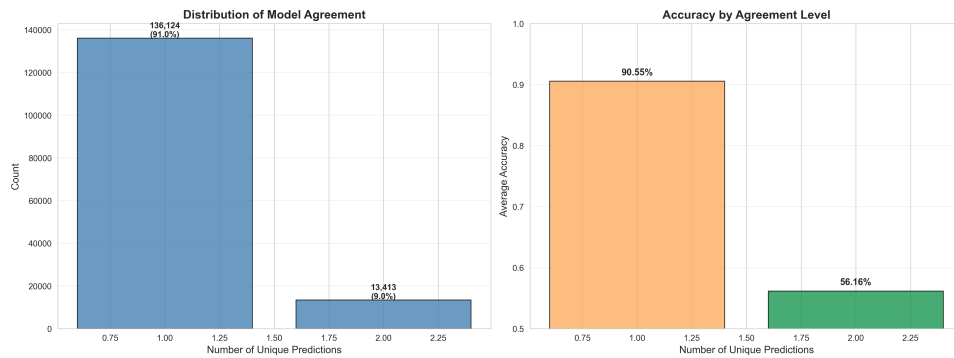


Figura 2.7: Le 15 tecniche MITRE ATT&CK più frequenti nel dataset.

2.4 Strumenti Utilizzati

Per l'analisi dei dati, il preprocessing e lo sviluppo dei modelli predittivi sono stati impiegati i seguenti strumenti, librerie e framework, scelti per la loro robustezza, flessibilità e ampia adozione nella comunità di data science.

- **Python:** Linguaggio di programmazione principale, scelto per il suo vasto ecosistema di librerie per l'analisi dei dati, machine e deep learning.
- **Pandas:** Libreria fondamentale per la manipolazione, la pulizia e l'analisi di dati strutturati. È stata utilizzata per caricare, trasformare e aggregare i dati del dataset GUIDE.
- **Scikit-learn:** La libreria di riferimento per l'implementazione di algoritmi di machine learning. È stata utilizzata per il preprocessing dei dati (scaling, encoding), l'addestramento dei modelli di classificazione (eccetto MLP) e la loro valutazione tramite metriche e tecniche di convalida incrociata.
- **PyTorch:** libreria per python che permette di costruire, addestrare e utilizzare modelli di deep learning, in questo caso usato per un modello di classificazione MLP.
- **Matplotlib & Seaborn:** Librerie utilizzate per la creazione di visualizzazioni statiche, come istogrammi, grafici a barre, matrici di correlazione e curve ROC, essenziali per la fase di analisi esplorativa e per la presentazione dei risultati.
- **Jupyter Notebook:** Ambiente di sviluppo interattivo che ha permesso di combinare codice, visualizzazioni e testo descrittivo, facilitando l'esplorazione dei dati e la documentazione del processo di analisi.

- **VSCode e PyCharm:** gli editor/ambienti di sviluppo utilizzati per l'implementazione del codice.

3 Extract - Transform - Load

Il preprocessing dei dati a disposizione è stato cruciale data l'elevata dimensionalità (oltre 40 colonne) ed eterogeneità degli stessi. L'obiettivo di questa fase è stato quello di ridurre e trasformare i dati grezzi di GUIDE in un formato pulito, coerente e ottimizzato per addestrare e testare gli algoritmi di classificazione. Le operazioni di pulizia permettono di massimizzare le performance dei modelli, riducendo il rischio di overfitting e garantendo che l'algoritmo apprenda pattern significativi anziché rumore statistico.

3.1 Preparazione del Dataset

Durante questa fase, si sono susseguite diverse operazioni per preparare i dati:

- **Estrazione e Aggregazione:** i dati sono stati aggregati a livello di `IncidentId`. Le informazioni provenienti da più avvisi ed entità sono state consolidate per creare un singolo record per ogni incidente, generando feature come `AlertCount`, `EntityCount` e aggregando le tecniche MITRE.
- **Gestione dei Valori Nulli:** il dataset è stato analizzato per individuare l'eventuale presenza di valori nulli. Le colonne con un'alta percentuale di valori mancanti sono state rimosse, mentre per altre sono state applicate strategie di imputazione, come la sostituzione con la moda per le variabili categoriali.
- **Feature Engineering:** sono state create nuove feature per arricchire il dataset. Ad esempio, dalla colonna `Timestamp` sono state estratte informazioni temporali come l'ora del giorno e il giorno della settimana. La lista di `MitreTechniques` è stata trasformata in feature numeriche (es. conteggio o one-hot encoding).
- **Conversione dei Tipi di Dati:** le colonne sono state convertite nei formati appropriati (es. da `object` a `datetime` per `Timestamp`, da

string a category per le variabili categoriali) per ottimizzare l'uso della memoria e facilitare le successive analisi.

3.2 Pulizia e Gestione dei Dati

Le decisioni prese durante la fase di pulizia sono state guidate dalla necessità di creare un dataset coerente e robusto, eliminando il rumore e gestendo le imperfezioni dei dati grezzi.

3.2.1 Eliminazione delle Colonne

Diverse colonne sono state identificate e rimosse in quanto non fornivano informazioni utili per il task di classificazione o avrebbero potuto introdurre rumore:

- **Identificatori Unici:** colonne come `IncidentId` e `AlertId` sono state rimosse. Essendo identificatori unici per ogni entry, hanno una cardinalità pari al numero di campioni e non offrono alcun pattern generalizzabile che un modello possa apprendere.
- **Colonne con Alta Percentuale di Valori Mancanti:** feature con una percentuale di valori nulli superiore a una soglia critica (97% in questo caso) sono state scartate, poiché qualsiasi tecnica di imputazione sarebbe stata inaffidabile e avrebbe potuto introdurre artefatti.
- **Feature a Varianza Nulla:** colonne i cui valori erano costanti per tutti i campioni sono state eliminate, in quanto non portano alcuna informazione discriminante.

3.2.2 Rinominazione delle Colonne

Per migliorare la leggibilità e la coerenza del codice, tutte le colonne sono state rinominate utilizzando lo standard `snake_case` (es. `TriageStatus` è diventato `triage_status`). Questa convenzione semplifica la manipolazione dei dati e rende il codice più manutenibile.

3.2.3 Gestione dei Valori Nulli

È stata condotta un'analisi dettagliata dei valori mancanti per ogni colonna rimanente. In base alla natura della variabile e alla distribuzione dei dati, sono state adottate le seguenti strategie:

- **Dati categorici:** per le colonne categoriche come `Roles`, `ActionGrouped`, `SuspicionLevel` e altre, i valori mancanti sono stati sostituiti con

una stringa costante come 'Missing' o 'missing' prima di applicare il frequency o one-hot encoding.

- **Dati numerici:** dopo l'aggregazione e prima della suddivisione in set di addestramento e test, eventuali valori mancanti rimanenti nella matrice delle feature X sono stati riempiti con il valore -999 tramite `X.fillna(-999)`.

3.3 Feature Engineering

Questa fase si è concentrata sulla creazione di nuove feature informative e sulla trasformazione di quelle esistenti in un formato numerico adatto agli algoritmi di machine learning, scartando quelle non utili ai fini dell'analisi.

3.3.1 Elaborazione della Colonna delle Tecniche MITRE

La colonna `mitre_techniques`, contenente una lista di tecniche MITRE ATT&CK per ogni incidente, rappresentava una fonte di informazione di grande valore. Per renderla utilizzabile, è stata applicata una tecnica di binarizzazione multi-etichetta. Dato l'elevato numero di tecniche uniche, per evitare un'esplosione della dimensionalità, sono state considerate solo le 30 tecniche più frequenti. È stato quindi utilizzato un `MultiLabelBinarizer` per creare 30 nuove colonne binarie, dove ogni colonna indica la presenza (1) o l'assenza (0) di una specifica tecnica in un dato incidente.

Infine, si passa da One-Hot encoding delle MITRE a un conteggio delle occorrenze delle stesse aggregandole per somma a livello `Incident`.

3.3.2 Codifica delle Variabili Categoriali

Le restanti feature categoriche sono state trasformate in rappresentazioni numeriche:

- **One-Hot Encoding:** per le variabili nominali con bassa cardinalità (es. `severity`), è stato utilizzato il One-Hot Encoding. Questa tecnica crea nuove colonne binarie per ogni categoria, evitando di introdurre un ordine artificiale tra i valori.
- **Target Encoding:** per le variabili con alta cardinalità come `detector_id`, il One-Hot Encoding sarebbe stato impraticabile. È stata quindi impiegata una tecnica di Target Encoding, dove ogni categoria viene sostituita dalla media della variabile target per quella categoria. Per prevenire il data leakage, questa operazione è stata eseguita all'interno di una pipeline di cross-validation.

3.3.3 Scaling delle Feature Numeriche

Infine, tutte le feature numeriche sono state scalate utilizzando lo `StandardScaler`. Questa trasformazione standardizza le feature portandole ad avere una media pari a 0 e una deviazione standard pari a 1. Lo scaling è fondamentale per le performance di molti algoritmi di machine learning, come la Regressione Logistica e le Support Vector Machines, che sono sensibili alla scala delle feature di input.

3.4 Il Problema dello Sbilanciamento

Una volta suddiviso il dataset risultante dalle precedenti fasi in training e test set, è stata controllata nuovamente la distribuzione delle due classi target.

La nuova analisi condotta sui dati di addestramento ha condotto alle seguenti identificazioni:

- Classe 0 (Non-TP) : 247,302 campioni, corrispondenti al 78.70% sul totale del Training Set.
- Classe 1 (TP) : 66,928 campioni, corrispondenti al 21.30% sul totale del Training Set.

I dati riportati evidenziano un pesante *class-imbalance*, con un rapporto di 3.7 : 1 in favore della classe 0.

3.4.1 Bilanciamento Manuale

Data la mole di dati disponibili si è preferito ricorrere all'utilizzo di campioni della classe minoritaria presenti nel `GUIDE_Test`. E' importante sottolineare che non sono stati utilizzati dati provenienti dal Test Set: tale dataset viene creato dinamicamente alla fine della pipeline di alimentazione.

- **Classe 1 in `GUIDE_Test`:** il numero di campioni è 49,554: saranno tutti utilizzati per il bilanciamento.
- **Risultati:** il nuovo dataset bilanciato contiene:
 - Classe 0 (Non-TP) : 247,302 campioni, corrispondenti al 70.88% sul totale del Training Set.
 - Classe 1 (TP) : 101,616 campioni, corrispondenti al 29.12% sul totale del Training Set.

Al termine di questa procedura è stato raggiunto un nuovo *class-imbalance ratio* pari a 2.43 : 1.

3.4.2 Sotto-campionamento e SMOTE

Poiché i risultati raggiunti nella fase precedente non permettevano ancora ai modelli di raggiungere un buon livello di F1-Score, ridotto dalla scarsa precisione ottenuta nella predizione delle classi minoritarie, sono state adottate ulteriori tecniche di bilanciamento:

- **Undersampling:** il sottocampionamento della classe maggioritaria è stato reso possibile grazie all'ampia disponibilità di dati, riducendo il numero di campioni a 152,424 e raggiungendo un *imbalance ratio* di 1.5 : 1.
- **SMOTE:** questa tecnica di oversampling è stata applicata in modo tale da ridurre ulteriormente questo divario, bilanciando completamente le classi.

Dopo aver testato separatamente le tecniche, è stata scelta come implementazione finale una pipeline che applica l'undersampling in un primo momento, seguito dallo SMOTE. I risultati così raggiunti saranno discussi nella sezione dedicata alla valutazione delle metriche.

3.5 Risultati della Fase di Preprocessing

Al termine di questo processo, il dataset grezzo è stato trasformato in una matrice di feature completamente numerica, pulita e pronta per la fase di modellazione. La dimensionalità del dataset è stata gestita attentamente, bilanciando l'inclusione di informazioni pertinenti con la necessità di evitare il "Curse of Dimensionality". La Figura 3.1 mostra la matrice di correlazione calcolata sul dataset finale, evidenziando l'assenza di forti correlazioni tra le features numeriche, un indicatore positivo per la stabilità dei modelli.

Le uniche evidenti correlazioni fanno riferimento a feature aggregate, suggerendo correlazioni spurie tra queste.

Il risultato finale è un set di dati robusto che pone solide fondamenta per l'addestramento e la valutazione di modelli predittivi efficaci, tenendo sempre presente la sfida posta dallo sbilanciamento delle classi, che verrà affrontata nel capitolo successivo.

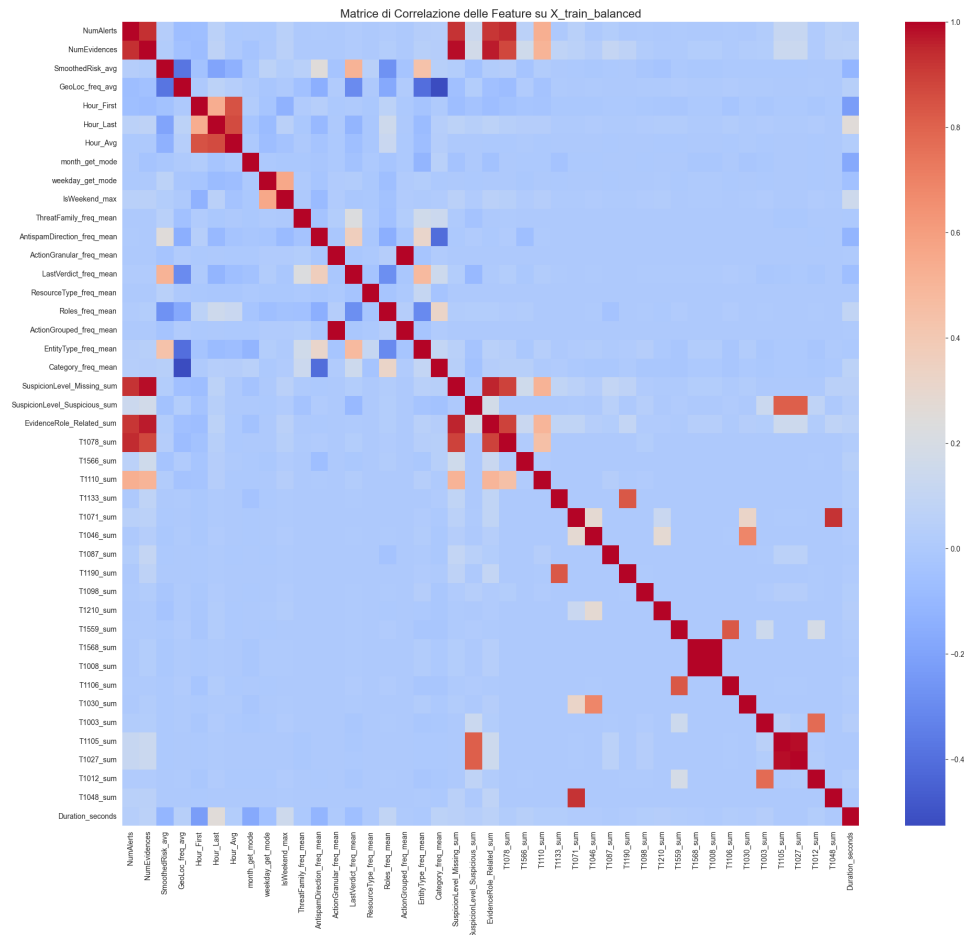


Figura 3.1: Matrice di correlazione (heatmap) tra le principali feature numeriche dopo il preprocessing. L'assenza di colori intensi (vicini a +1 o -1) indica una bassa correlazione, suggerendo che le feature portano informazioni relativamente indipendenti.

4 Classificazione

Una volta ottenuto un dataset pulito, strutturato e non eccessivamente numeroso, la fase successiva e centrale del progetto si è concentrata sullo sviluppo, l'ottimizzazione e la valutazione rigorosa di diversi modelli di Intelligenza Artificiale.

L'obiettivo di questo capitolo è descrivere il processo di selezione degli algoritmi, confrontare sistematicamente le loro performance e analizzare in profondità i risultati per identificare l'approccio più efficace per la classificazione automatica del triage degli incidenti di sicurezza.

4.1 Metodologia di Valutazione

Per garantire una valutazione oggettiva e robusta, è stata definita una metodologia standard applicata a tutti i modelli.

- **Suddivisione del Dataset:** Il dataset è stato suddiviso in un set di addestramento (80%) e un set di test (20%). Tutti i processi di addestramento e ottimizzazione sono stati eseguiti esclusivamente sul set di addestramento, mentre il set di test è stato tenuto da parte come "hold-out set" per la valutazione finale e imparziale del modello.
- **Validazione Incrociata:** Per l'ottimizzazione degli iperparametri, è stata utilizzata una strategia di validazione incrociata a 5 fold (*5-fold cross-validation*) per ottenere una stima stabile delle performance e ridurre il rischio di overfitting.
- **Metriche di Valutazione:** Data la natura sbilanciata del dataset, l'accuratezza da sola non è una metrica sufficiente. La valutazione si è concentrata su un insieme di metriche più informative: **Precisione**, **Recall**, **F1-Score** e **ROC AUC**. La metrica principale per il confronto e l'ottimizzazione è stata il **Macro F1-Score**, che calcola la media dell'F1-score per ogni classe senza ponderazione, dando uguale importanza alla performance su classi minoritarie e maggioritarie.

4.2 Modelli di Classificazione

Data la natura del task e la numerosità del dataset, sono stati scartati i modelli della famiglia "Nearest - Neighbor": sarebbe infatti risultato difficile e computazionalmente oneroso riuscire ad utilizzarli con oltre 400 mila record.

Poiché l'obiettivo dei modelli è quello di distinguere un rischio reale da un falso allarme, il numero di categorie è stato ridotto da 3 a 2, accorpendo "bening_positive" e "false_positive" in un'unica label.

Questa trasformazione ha modificato la natura della classificazione, che da multiclasse è divenuta binaria, aumentando di conseguenza anche le prestazioni dei modelli stessi.

Dunque le scelte sono ricadute su tre tipologie di modelli: *Decision Tree*, *Random Forest/XGBoost* e *Multi-Layer Perceptron*.

4.2.1 Decision Tree

L'Albero Decisionale è un algoritmo di classificazione basato sullo split iterativo dei dati in funzione delle feature ed è stato implementato attraverso la libreria *sklearn*. Fin dai primi test ha dimostrato prestazioni elevate sul test set (comune a tutti i modelli).

```
1     model = DecisionTreeClassifier(  
2         max_depth=10,  
3         min_samples_split=100,  
4         min_samples_leaf=50,  
5         class_weight='balanced',  
6         random_state=42  
7     )  
8  
9     model.fit(X_train, y_train)
```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri assegnati al modello, in particolare:

- **max_depth**: impostata a 10, utile per limitare la profondità dell'albero, riduce il rischio di overfitting.
- **min_samples_split**: minimo sample per split.
- **min_samples_leaf**: minimo sample per foglia.

- **class_weight:** impostato come "balanced" per utilizzare il bilanciamento automatico dei pesi delle classi, inizialmente necessario a causa del *class_imbalance* poi eliminato con undersampling e SMOTE.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.1.

	precision	recall	f1-score	support
Non-TP	0.9000	0.9204	0.9101	105987
TP	0.7950	0.7511	0.7724	43550
accuracy			0.8711	149537
macro avg	0.8475	0.8358	0.8413	149537
weighted avg	0.8694	0.8711	0.8700	149537

PRECISION: 0.7950

RECALL: 0.7511

F1-SCORE: 0.7724

ROC AUC: 0.9224

Figura 4.1: Metriche di classificazione dell'albero decisionale.

Un ulteriore punto di vista delle prestazioni del modello può essere fornito dalla matrice di confusione tra label reale e predetta di figura 4.1 e la curva ROC di figura 4.3.

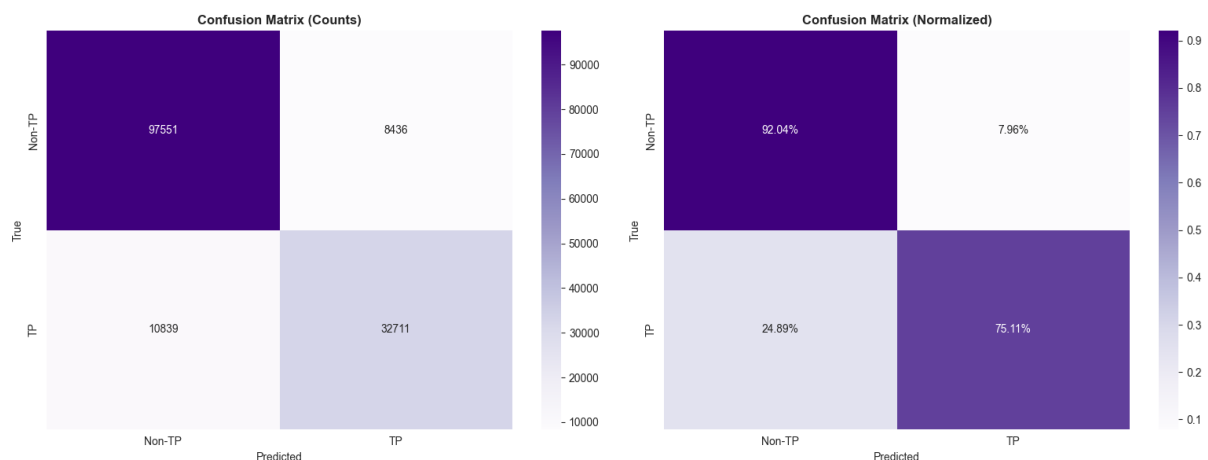


Figura 4.2: Matrice di confusione dell'albero decisionale.

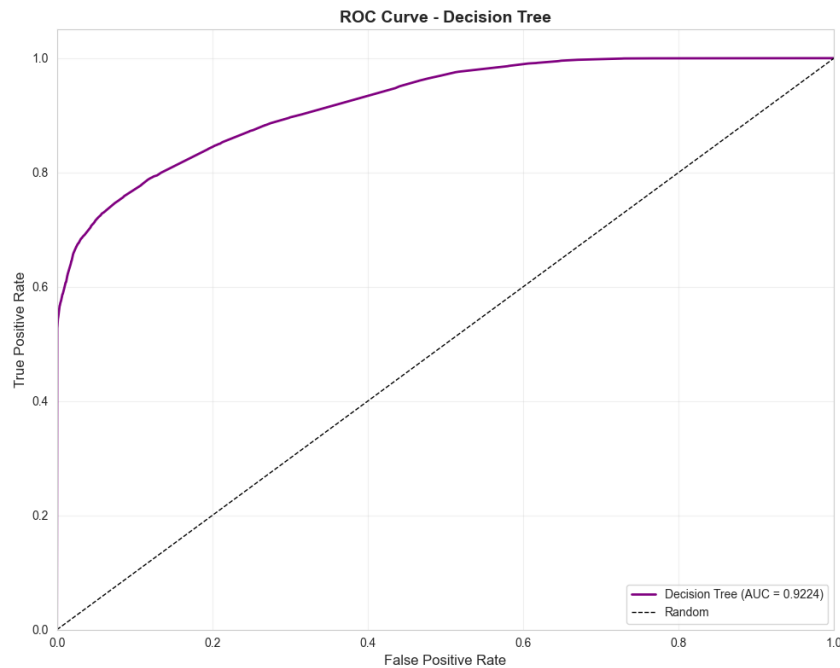


Figura 4.3: Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).

4.2.2 XGBoost

L'XGBoost (eXtreme Gradient Boosting) è un algoritmo di machine learning basato sull'idea del boosting, in cui vengono creati in sequenza dei modelli deboli (solitamente alberi decisionali) in modo che ogni nuovo modello corregga gli errori del precedente. È noto per le sue elevate prestazioni e la sua velocità di esecuzione, ed è stato implementato tramite la libreria `xgboost`.

```
1 model = xgb.XGBClassifier(  
2     objective='binary:logistic',  
3     eval_metric='auc',  
4     max_depth=6,  
5     learning_rate=0.1,  
6     n_estimators=200,  
7     subsample=0.8,  
8     colsample_bytree=0.8,  
9     scale_pos_weight=scale_pos_weight,  
10    random_state=42,  
11    tree_method='hist',  
12 )  
13  
14 model.fit(X_train, y_train)
```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri assegnati al modello, in particolare:

- **objective:** impostato su 'binary:logistic' per problemi di classificazione binaria.
- **n_estimators:** il numero di alberi (passaggi di boosting) da eseguire.
- **max_depth:** la profondità massima di un albero per controllare la complessità.
- **learning_rate:** il tasso di apprendimento, che riduce il contributo di ciascun albero per prevenire l'overfitting.
- **subsample:** la frazione di campioni da utilizzare per l'addestramento di ciascun albero.
- **colsample_bytree:** la frazione di feature da utilizzare per la costruzione di ciascun albero.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.4.

	precision	recall	f1-score	support
Non-TP (0)	0.9016	0.9303	0.9157	105987
TruePositive (1)	0.8161	0.7530	0.7833	43550
accuracy			0.8786	149537
macro avg	0.8588	0.8416	0.8495	149537
weighted avg	0.8767	0.8786	0.8771	149537
ACCURACY:	0.8786			
PRECISION:	0.8161			
RECALL:	0.7530			
F1-SCORE:	0.7833			
ROC AUC:	0.9307			

Figura 4.4: Metriche di classificazione di XGBoost.

Un ulteriore punto di vista delle prestazioni del modello può essere fornito dalla matrice di confusione tra label reale e predetta di figura 4.5 e la curva ROC di figura 4.6.

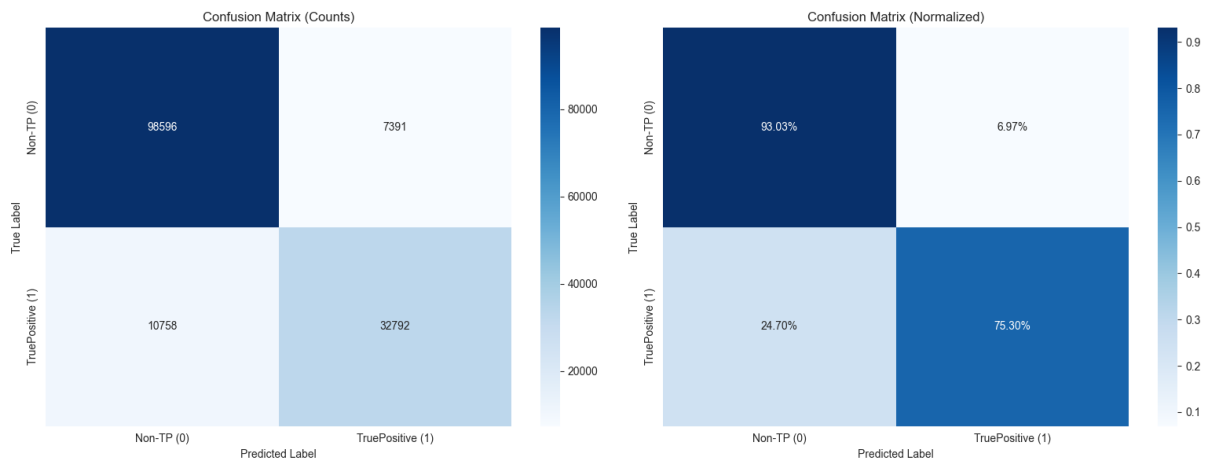


Figura 4.5: Matrice di confusione di XGBoost.

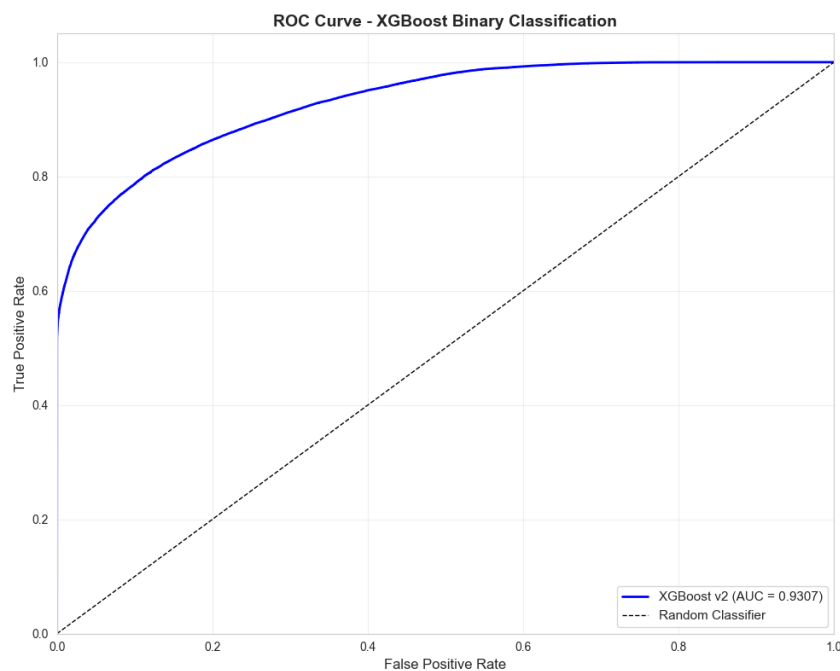


Figura 4.6: Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).

4.2.3 Random Forest

La Random Forest è un algoritmo di *ensemble learning* che costruisce una moltitudine di alberi decisionali in fase di training e restituisce la classe che è la moda delle classi dei singoli alberi. È efficace nel ridurre l'overfitting rispetto a un singolo albero decisionale ed è stato implementato attraverso la libreria `sklearn`.

```

1 model = RandomForestClassifier(
2     n_estimators=100,
3     max_depth=10,
4     min_samples_split=100,

```

```

5         min_samples_leaf=50,
6         class_weight='balanced',
7         random_state=42
8     )
9
10    model.fit(X_train, y_train)

```

Nella porzione di codice riportata nel paragrafo precedente sono evidenti i parametri assegnati al modello, in particolare:

- **n_estimators**: il numero di alberi nella foresta.
- **max_depth**: la profondità massima di ogni albero.
- **min_samples_split**: il numero minimo di campioni richiesto per split-tare un nodo interno.
- **min_samples_leaf**: il numero minimo di campioni richiesto per essere in un nodo foglia.
- **class_weight**: impostato come "balanced" per assegnare un peso alle classi inversamente proporzionale alla loro frequenza.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.7.

	precision	recall	f1-score	support
Non-TP	0.8925	0.9472	0.9190	105987
TP	0.8489	0.7223	0.7805	43550
accuracy			0.8817	149537
macro avg	0.8707	0.8347	0.8497	149537
weighted avg	0.8798	0.8817	0.8787	149537
PRECISION:	0.8489			
RECALL:	0.7223			
F1-SCORE:	0.7805			
ROC AUC:	0.9241			

Figura 4.7: Metriche di classificazione della Random Forest.

Un ulteriore punto di vista delle prestazioni del modello può essere fornito dalla matrice di confusione tra label reale e predetta di figura 4.8 e la curva ROC di figura 4.9.

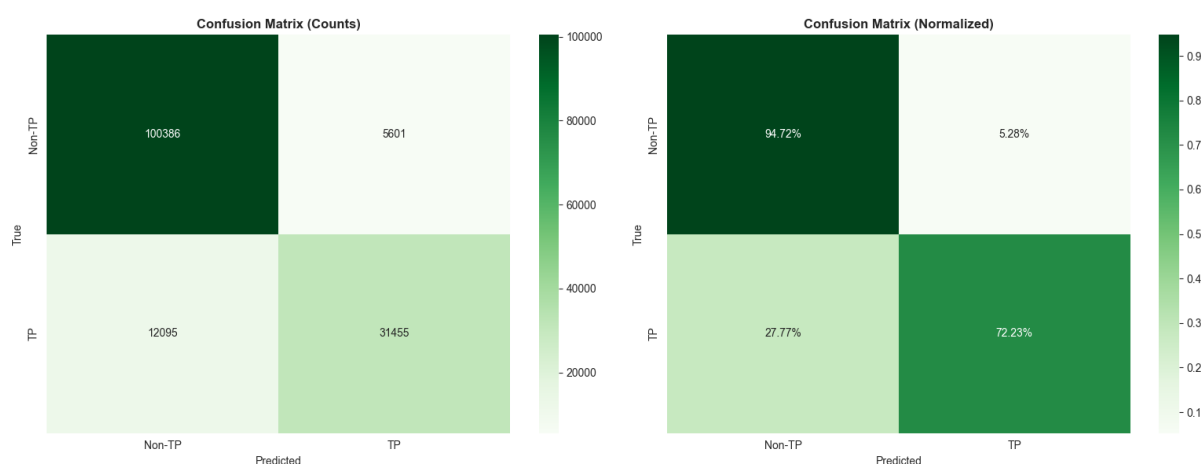


Figura 4.8: Matrice di confusione della Random Forest.

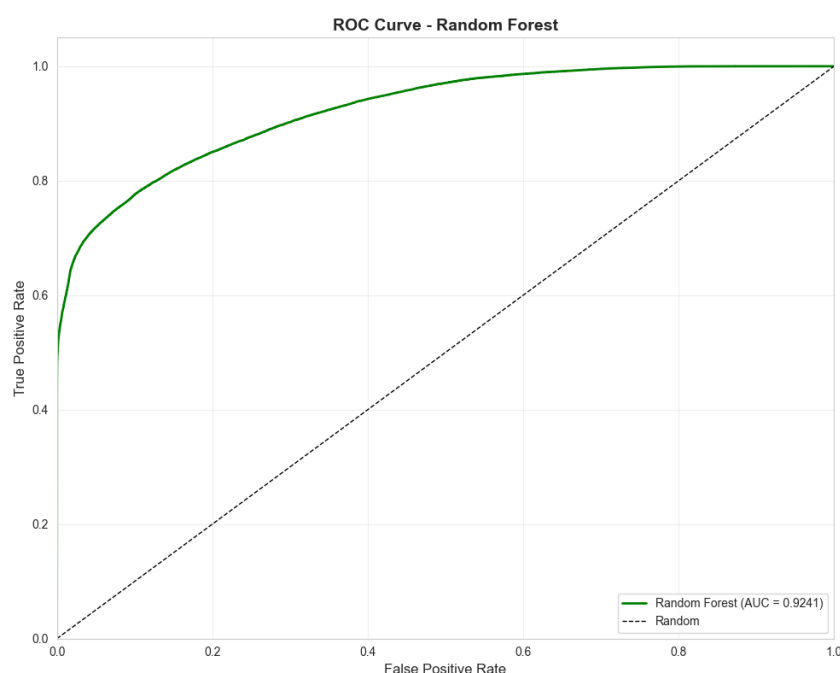


Figura 4.9: Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).

4.2.4 Multi-layer Perceptron

Il Multi-layer Perceptron (MLP) è una classe di rete neurale artificiale *feed-forward*, implementata in questo caso attraverso la libreria PyTorch per un controllo più granulare sull'architettura. Un MLP è composto da almeno tre strati di nodi: uno strato di input, uno o più strati nascosti (hidden layers) e uno strato di output. L'apprendimento avviene tramite l'algoritmo di bac-

kpropagation, che aggiorna i pesi della rete per minimizzare una funzione di costo.

```
1 import torch.nn as nn
2
3 class MLP(nn.Module):
4     def __init__(self, input_dim, dropout=0.3):
5         super(MLP, self).__init__()
6         self.network = nn.Sequential(
7             nn.Linear(input_dim, 128),
8             nn.ReLU(),
9             nn.BatchNorm1d(128),
10            nn.Dropout(dropout),
11
12            nn.Linear(128, 64),
13            nn.ReLU(),
14            nn.BatchNorm1d(64),
15            nn.Dropout(dropout),
16
17            nn.Linear(64, 32),
18            nn.ReLU(),
19            nn.BatchNorm1d(32),
20            nn.Dropout(dropout),
21
22            nn.Linear(32, 1)
23        )
24
25    def forward(self, x):
26        return self.network(x)
```

L'architettura definita nel codice precedente è composta da una sequenza di strati (layers), ognuno con uno scopo specifico:

- **Architettura:** la rete è formata da quattro strati lineari (`nn.Linear`) che riducono progressivamente la dimensionalità dei dati da `input_dim` a 128, poi a 64, 32 e infine a 1. L'output finale è un singolo valore (logit), adatto per la classificazione binaria.
- **Funzione di Attivazione:** dopo i primi tre strati lineari viene applicata la funzione `nn.ReLU` (Rectified Linear Unit), che introduce non-linearità nel modello, permettendogli di apprendere relazioni più complesse.
- **Normalizzazione:** lo strato `nn.BatchNorm1d` (Batch Normalization) viene utilizzato per normalizzare l'output dello strato precedente. Que-

sto stabilizza il processo di apprendimento e accelera la convergenza.

- **Regolarizzazione:** per prevenire l'overfitting, è stato inserito uno strato di `nn.Dropout` dopo ogni blocco di attivazione e normalizzazione. Durante l'addestramento, questo strato "spegne" casualmente una frazione dei neuroni (in questo caso il 30%), costringendo la rete a imparare feature più robuste.

Al termine del training sull'ultimo dataset sono stati ottenuti i valori delle metriche riportati nella tabella di figura 4.10.

	precision	recall	f1-score	support
Non-TP	0.8977	0.9177	0.9076	105987
TP	0.7881	0.7454	0.7662	43550
accuracy			0.8675	149537
macro avg	0.8429	0.8315	0.8369	149537
weighted avg	0.8658	0.8675	0.8664	149537

PRECISION: 0.7881
RECALL: 0.7454
F1-SCORE: 0.7662
ROC AUC: 0.9200

Figura 4.10: Metriche di classificazione del Multi-layer Perceptron.

Un ulteriore punto di vista delle prestazioni del modello può essere fornito dalla matrice di confusione tra label reale e predetta di figura 4.11 e la curva ROC di figura 4.12.

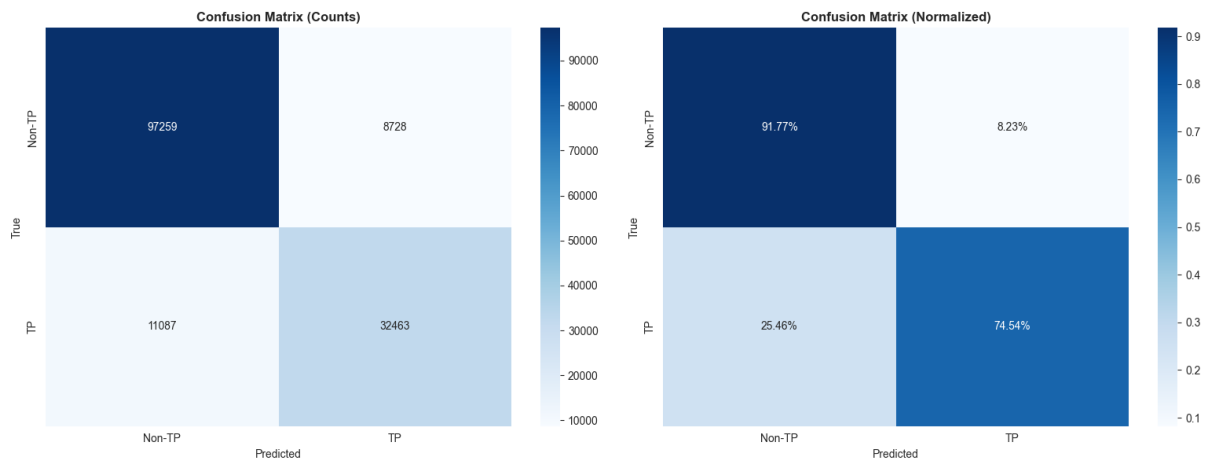


Figura 4.11: Matrice di confusione del Multi-layer Perceptron.

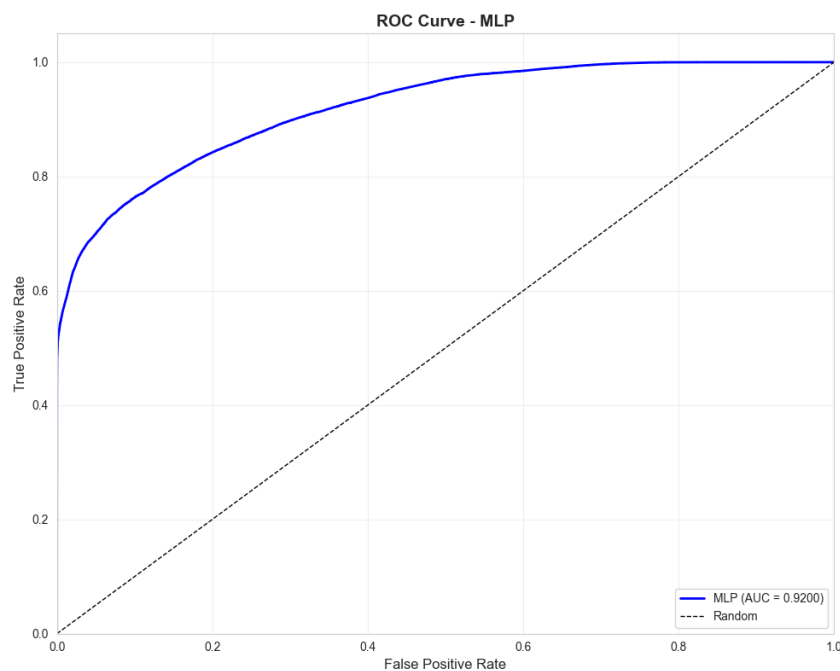


Figura 4.12: Curva ROC (prestazioni migliori se la curva è vicina all'angolo alto a sinistra).

Considerazioni Operative

Per velocizzare le operazioni di training e inferenza del modello MLP è stato implementato un sistema di safe-fallback dove a seconda dell'hardware che si sta utilizzando sceglie il dispositivo corretto da utilizzare.

- **CUDA**: per i sistemi dotati di schede grafiche Nvidia con cuDNN 12.1 e compatibili. In questo caso è stata utilizzata una RTX 3060Ti con 8GB di VRAM.
- **MPS**: specifico per sistemi Apple con hardware AMD Radeon e M-Series, testato su M4 Pro.

- **CPU**: ultimo stadio del fallback, utilizza la CPU come dispositivo per il training (e l'inferenza) solo nel caso in cui non siano disponibili i due precedenti. È importante sottolineare che si avrà un'elaborazione molto più lenta.

Pertanto, si suggerisce di utilizzare CUDA o MPS per replicare i risultati ottenuti in questa trattazione.

4.3 Valutazione Comparativa dei Modelli

Sono stati addestrati e valutati quattro diversi algoritmi: Logistic Regression (come baseline), Decision Tree, Random Forest e XGBoost. La Figura 4.13 offre una panoramica completa delle loro performance su tutte le metriche chiave.

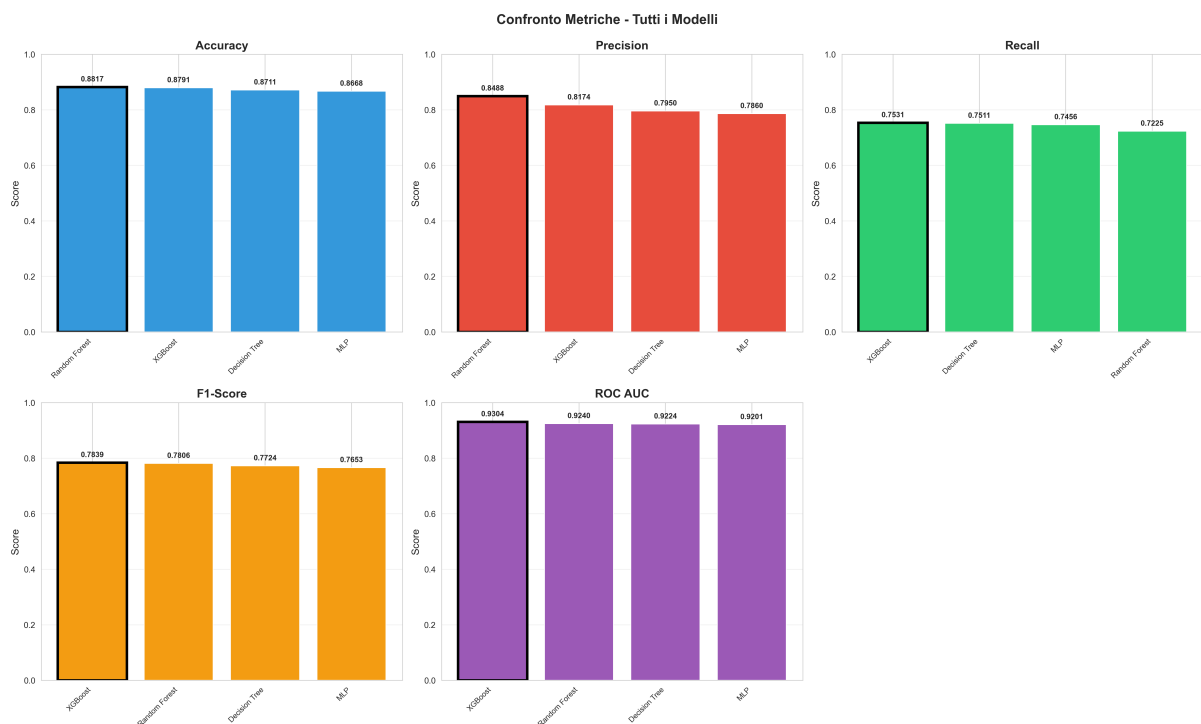


Figura 4.13: Confronto delle performance dei quattro modelli su cinque metriche chiave: Accuracy, Precision, Recall, F1-Score e ROC AUC. Si osserva come i modelli basati su alberi (Random Forest, XGBoost) tendano a sovraperformare il modello MLP di base.

Come si evince dal confronto, i modelli basati su ensemble di alberi (Random Forest e XGBoost) mostrano una superiorità generale. Per approfondire, la Figura 4.14 si concentra sulla metrica ROC AUC, che misura la capacità di un modello di distinguere tra le classi.

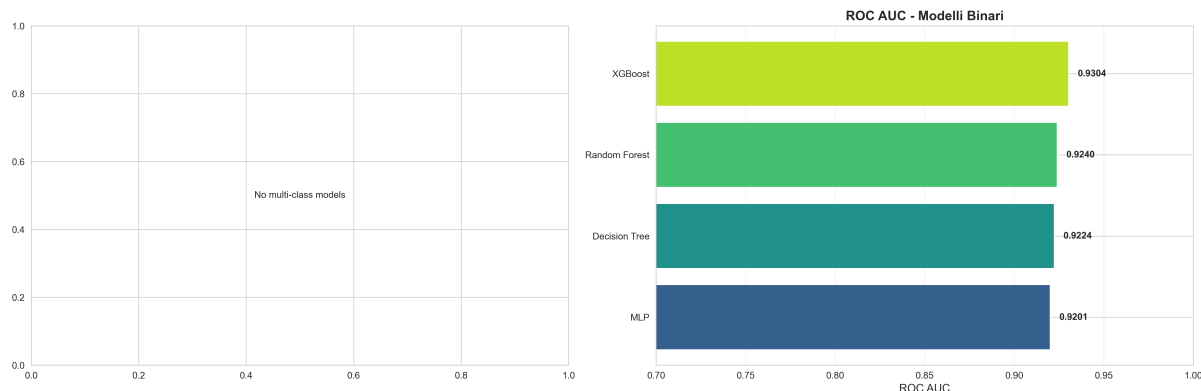


Figura 4.14: Confronto dei punteggi ROC AUC per i modelli binari. XGBoost emerge come il modello con la più alta capacità discriminante (AUC = 0.9304), seguito da vicino da Random Forest e Decision Tree.

4.4 Analisi Dettagliata delle Performance

Per comprendere non solo quanto bene i modelli performano, ma anche come commettono errori, sono state analizzate le matrici di confusione. La Figura 4.15 confronta il comportamento di tre dei modelli principali.

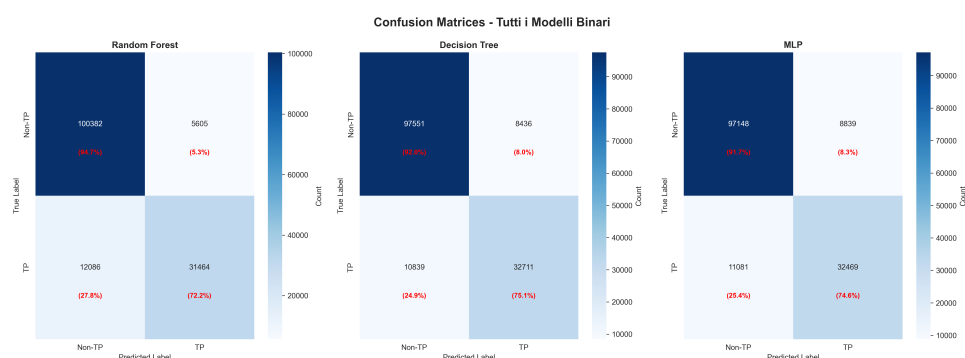


Figura 4.15: Matrici di confusione per i modelli Random Forest, Decision Tree e MLP. I valori indicano il numero di predizioni corrette e scorrette. La percentuale di recall per ogni classe ($TP / (TP + FN)$) è indicata in rosso. Si nota come tutti i modelli siano molto efficaci nell'identificare i casi Non-TP, ma il Random Forest mostra il miglior recall sulla classe TP (72.2%).

4.5 Analisi di Interpretabilità: Feature Importance

Capire quali feature influenzano maggiormente le decisioni di un modello è cruciale per guadagnare fiducia nei suoi risultati e per ottenere insight sul fenomeno studiato. La Figura 4.16 confronta le 10 feature più importanti per i tre modelli basati su alberi.

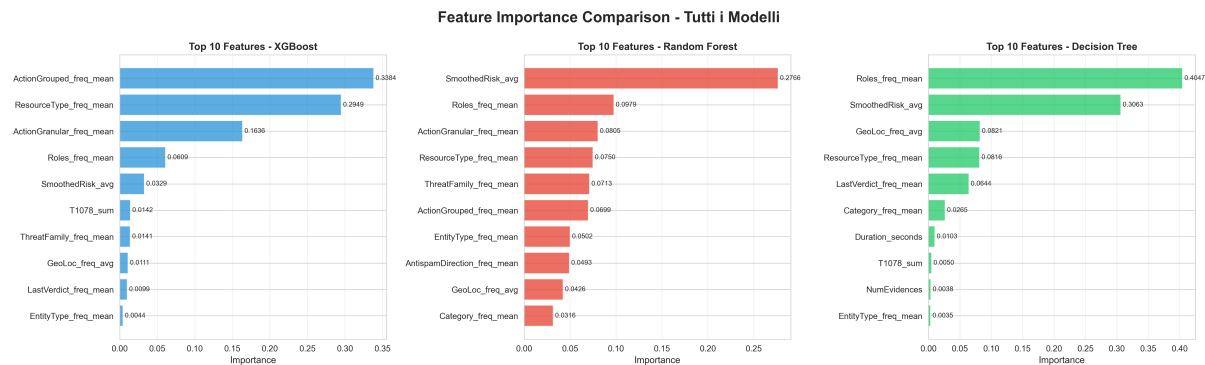


Figura 4.16: Confronto delle 10 feature più importanti secondo i modelli XGBoost, Random Forest e Decision Tree. Sebbene ogni modello dia un peso leggermente diverso, emergono pattern comuni.

Per sintetizzare questi risultati, la Figura 4.17 identifica le feature che appaiono costantemente tra le più importanti in tutti i modelli, evidenziandole come i predittori più robusti e affidabili nel nostro dataset.

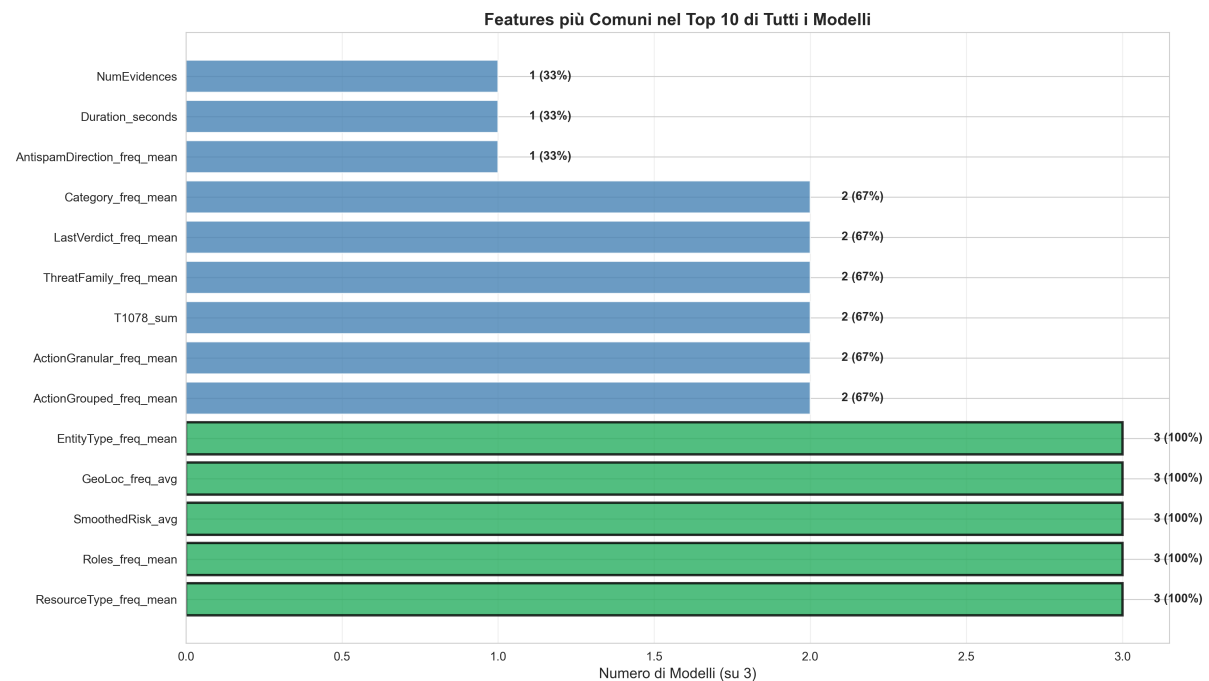


Figura 4.17: Analisi delle feature più comuni nella Top 10 di tutti e tre i modelli a albero.

4.6 Ensemble: analisi dell'Accordo tra Modelli

Solitamente, le tecniche di ensemble funzionano meglio quando i modelli base sono diversi e commettono errori non correlati. La Figura 4.18 mostra la matrice di correlazione tra le predizioni dei diversi modelli.

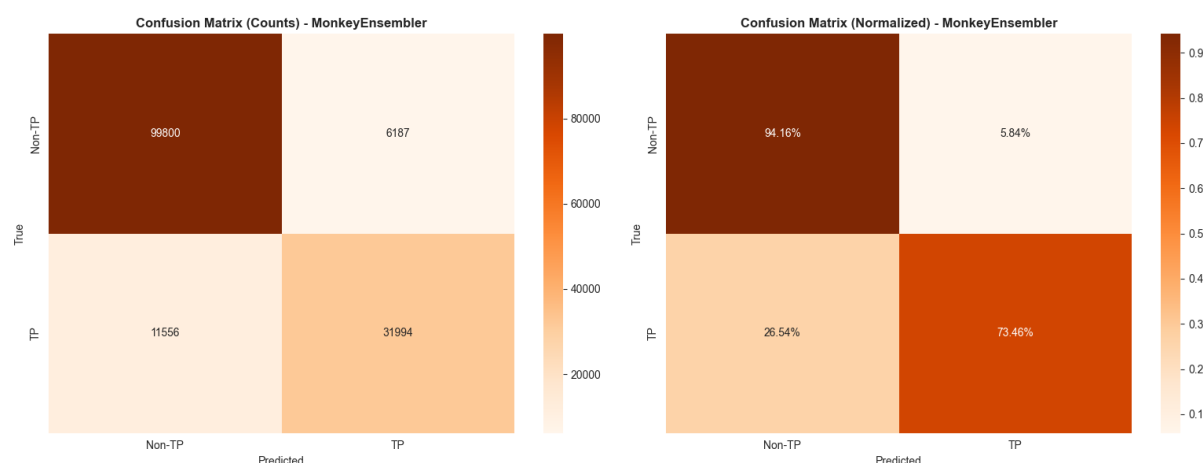


Figura 4.18: Matrice di correlazione tra le predizioni dei modelli.

Da un'altra analisi emerge che per l'F1-Score, il modello XGBoost (0.7833) e il Monkey Ensembler (0.7829) ottengono risultati molto simili e superiori a quelli del Random Forest (0.7805). Per quanto riguarda la metrica ROC AUC, il modello XGBoost si dimostra il più performante con un punteggio di 0.9307, seguito da vicino dal Monkey Ensembler (0.9298) e infine dal Random Forest (0.9241).

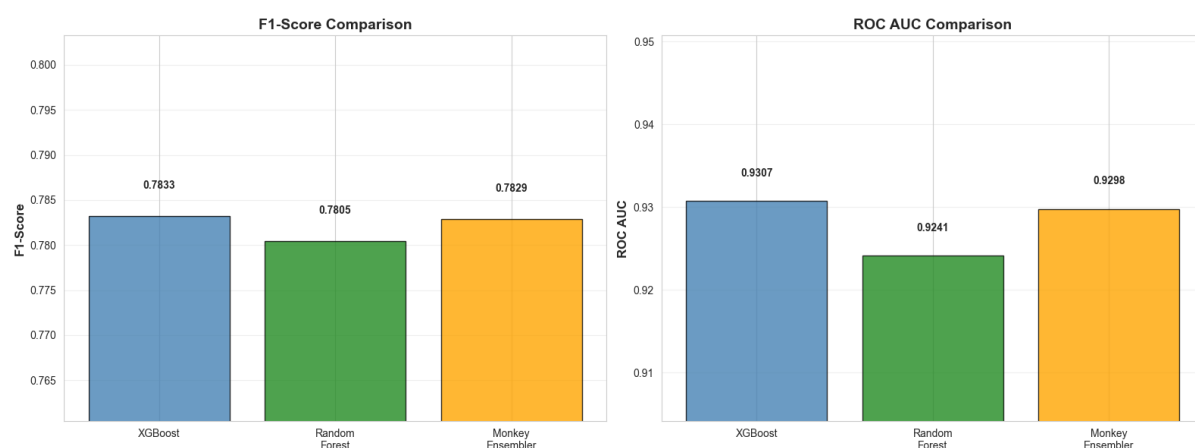


Figura 4.19: Confronto delle performance dei modelli in termini di F1-Score e ROC AUC.

4.6.1 Modelli Ensemble: XGBoost & Random Forest e XGBoost & MLP

Prendendo i modelli più promettenti, alla luce delle considerazioni del paragrafo precedente, è stato implementato un sistema di ensemble: per le

predizioni dell'XGBoost con confidenza inferiore ad un certo valore, sarà utilizzato il Random Forest.

Dopo diversi test, il miglioramento ottenuto con questa tecnica è stato di +0.15% sulla F1 - Score per le predizioni con confidenza minori del 52%. Anche utilizzando il modello che mette insieme XGBoost e MLP utilizzando una strategia di *soft voting* non si ottengono valori migliori, anzi: le metriche peggiorano di diversi punti percentuali.

Il risultato raggiunto non suggerisce una fattibile applicazione del metodo e di conseguenza si è deciso di non proseguire lungo questa strada.

```
📊 Performance Finali (Strategia: RF Override):  
ROC AUC: 0.9305  
F1-Score: 0.7844  
Accuracy: 0.8803
```

```
🆚 Confronto Strategie:  
RF Override: F1=0.7844, AUC=0.9305  
Soft Voting: F1=0.7820, AUC=0.9288
```

Figura 4.20: Valori finali delle metriche ensemble

4.7 Valutazione Finale del Modello

Sulla base dell'analisi precedente, il modello finale scelto è XGBoost, così come è stato definito nel rispettivo paragrafo. Questo modello si è dimostrato superiore, sebbene con un margine sottile, rispetto agli altri candidati, in particolare rispetto alla Random Forest che ha mostrato performance quasi altrettanto valide. La scelta si fonda su diverse considerazioni chiave emerse durante la valutazione comparativa:

- **Performance superiori:** come evidenziato in figura 4.19, XGBoost ha ottenuto il punteggio più alto in due delle metriche più significative per il nostro task. Ha mostrato il miglior **F1-Score** (0.7833), confermando un equilibrio ottimale tra precisione e recall, fondamentale per minimizzare sia i falsi positivi che i falsi negativi. Inoltre, ha raggiunto il miglior **ROC AUC score** (0.9307), indicando una capacità discriminante superiore nel distinguere correttamente tra le due classi.
- **Efficienza e scalabilità:** XGBoost è noto per la sua implementazione ottimizzata che garantisce alta velocità di addestramento e inferen-

za. Sebbene non sia stato un fattore critico con il dataset attuale, la sua efficienza lo rende una scelta più robusta e scalabile per future applicazioni dove il volume dei dati potrebbe aumentare significativamente.

- **Gestione dell'overfitting:** grazie a tecniche di regolarizzazione integrate (come L1 e L2), XGBoost offre un controllo più granulare contro l'overfitting rispetto ad altri modelli basati su alberi. Questo contribuisce a una maggiore generalizzazione delle performance su dati non visti.
- **Interpretabilità:** nonostante la sua complessità, XGBoost fornisce meccanismi robusti per l'analisi dell'importanza delle feature (come mostrato in figura 4.16), permettendo di comprendere quali variabili influenzano maggiormente le predizioni del modello. Questa trasparenza è cruciale per generare fiducia nel modello e per ottenere insight pratici dal processo di classificazione.
- **Inefficacia degli ensemble complessi:** i tentativi di creare un modello ensemble, combinando XGBoost con Random Forest o MLP, non hanno prodotto miglioramenti significativi da giustificare l'aumento di complessità. Anzi, in alcuni casi le performance sono addirittura peggiorate. Questo ha ulteriormente rafforzato la decisione di optare per il singolo modello più performante.

Dunque, XGBoost si è rivelato il miglior compromesso tra accuratezza predittiva, robustezza e interpretabilità per il task di classificazione in esame. La sua capacità di ottenere le migliori performance sulle metriche chiave, unita alla sua efficienza computazionale, lo rende la scelta ottimale per l'implementazione finale del sistema.

5 Conclusioni

Questo progetto ha affrontato la sfida complessa e di fondamentale importanza della classificazione automatica degli incidenti di cybersecurity. In un contesto operativo dove i Security Operations Centers (SOC) sono sommersi da un volume insostenibile di avvisi, l'automazione intelligente del processo di triage non è più un'opzione, ma una necessità strategica. Attraverso un'analisi approfondita del dataset su larga scala GUIDE e l'applicazione di una rigorosa pipeline metodologica di machine learning, sono stati sviluppati e validati modelli predittivi in grado di assistere efficacemente gli analisti, migliorando l'efficienza e l'efficacia della risposta agli incidenti.

5.1 Riepilogo e Interpretazione dei Risultati

Il modello finale sviluppato, XGBoost, ha dimostrato una notevole capacità nel predire le etichette di triage, raggiungendo un **F1-Score di 0.7833**. Questo risultato indica un solido equilibrio tra la capacità di identificare correttamente le minacce reali (TP) e di filtrare il rumore (FP), con una buona gestione anche della classe intermedia (BP). Le tecniche di ensemble non si sono rivelate particolarmente efficaci, rispetto al singolo XGBoost che aveva già ottenuto eccellenti risultati.

5.1.1 La Sfida dei "Benign Positive"

L'analisi delle performance ha evidenziato in modo inequivocabile che la classificazione degli incidenti "benign positive" (BP) rappresenta la sfida maggiore. Questo risultato non è sorprendente, ma è una conferma empirica della loro natura intrinsecamente ambigua. Gli incidenti BP rappresentano attività che, sebbene non dannose (come uno script di amministrazione legittimo eseguito in un orario insolito), generano pattern anomali che possono condividere caratteristiche sia con i veri positivi (TP) che con i falsi positivi (FP). Per queste ragioni sono stati aggregati con i falsi positivi, al fine di migliorare la discriminazione da parte del modello utilizzato. Comunque sia, la loro corretta identificazione rimane un'area chiave

per futuri miglioramenti e richiederà probabilmente l'ingegnerizzazione di feature più sofisticate o l'uso di modelli in grado di catturare un contesto ancora più ricco. In questo senso, utilizzare un modello Deep Learning in grado di catturare relazioni complesse potrebbe essere la soluzione, già il MLP utilizzato in questa trattazione ha raggiunto risultati eccellenti, seppur inferiori all'XGBoost.

5.1.2 Dinamicità del Dominio e Concept Drift

Una considerazione critica per l'applicazione pratica di questo lavoro è la natura dinamica della cybersecurity. Le tattiche, tecniche e procedure (TTP) degli avversari evolvono costantemente per eludere i sistemi di difesa. Ciò implica che qualsiasi modello predittivo, addestrato su dati storici, è inevitabilmente soggetto a "concept drift": il suo potere predittivo degraderà nel tempo man mano che emergono nuove minacce non viste in precedenza. Per questo motivo, un sistema di classificazione di questo tipo non può essere statico. La sua implementazione in un ambiente di produzione richiederebbe un framework di MLOps (Machine Learning Operations) che includa un monitoraggio continuo delle performance e un meccanismo di riaddestramento periodico con dati recenti e annotati, garantendo così la sua efficacia nel lungo periodo.

5.2 Contributi Principali del Progetto

Questo progetto ha permesso di esplorare un complesso dataset di cybersecurity del mondo reale e di applicare un flusso di lavoro completo di Data Science, fornendo diversi contributi significativi:

- La progettazione e l'implementazione di una **pipeline di preprocessing robusta e riproducibile**, in grado di gestire dati di sicurezza eterogenei e rumorosi, inclusa l'elaborazione di feature complesse e multi-label come le tecniche del framework MITRE ATT&CK.
- Lo **sviluppo e il confronto sistematico** di diversi modelli di classificazione, che ha permesso di validare empiricamente come gli approcci di ensemble basati su alberi decisionali (Random Forest, LightGBM) siano superiori ai modelli lineari per questo tipo di task.
- Il raggiungimento di un **solido benchmark di performance** sul dataset pubblico GUIDE. Questo risultato può servire come un valido punto di riferimento per futuri lavori di ricerca e per la valutazione di nuove tecniche sulla stessa base dati.

- L'analisi descrittiva e interpretativa delle feature, ha fornito **insight qualitativi** su quali caratteristiche (es. specifici `DetectorId`, combinazioni di tecniche MITRE, o la gravità degli avvisi) siano più predittive di una minaccia reale, informazioni preziose per gli analisti di sicurezza.

5.3 Sviluppi Futuri e Prospettive di Ricerca

Sebbene i risultati ottenuti siano incoraggianti, esistono ampi e promettenti margini di miglioramento. Le direzioni future per la ricerca potrebbero includere:

- **Deep Learning per Dati Sequenziali:** Gli incidenti di sicurezza sono spesso una sequenza di eventi. L'esplorazione di architetture di deep learning come le Reti Neurali Ricorrenti (RNN) o i Transformer potrebbe permettere di modellare le dipendenze temporali tra gli avvisi all'interno di un incidente, catturando pattern che i modelli statici non possono vedere.
- **Graph Neural Networks (GNNs):** La struttura gerarchica del dataset (prove → avvisi → incidenti) si presta naturalmente a una rappresentazione a grafo. L'utilizzo di Graph Neural Networks potrebbe rivelarsi estremamente potente per modellare esplicitamente le complesse relazioni tra le entità, apprendendo non solo dalle loro feature, ma anche dalla topologia delle loro connessioni.
- **Integrazione di Threat Intelligence Esterna:** Arricchire il dataset con fonti di dati esterne, come feed di threat intelligence su indirizzi IP malevoli, hash di file noti o domini di phishing, potrebbe fornire un contesto cruciale e migliorare significativamente la capacità del modello di identificare minacce emergenti.
- **Sistemi Human-in-the-Loop:** Invece di un'automazione completa, un'area di ricerca promettente è lo sviluppo di sistemi "human-in-the-loop", dove il modello fornisce raccomandazioni e probabilità all'analista, e allo stesso tempo impara dal feedback e dalle correzioni fornite da quest'ultimo. Questo approccio sinergico consentirebbe di ottenere dati estremamente precisi per un nuovo addestramento.

In conclusione, questo lavoro conferma il potenziale del machine learning come strumento per potenziare le capacità dei SOC. I modelli sviluppati rappresentano un passo concreto verso la realizzazione di sistemi di "risposta guidata" intelligenti, in grado di automatizzare il triage di routine e di consentire agli analisti umani di concentrare la loro insostituibile expertise sulle minacce più sofisticate e critiche.